PORTFOLIO PROJECT 02 / FULL-STACK + APPLIED AI

ResolveFlow AI

Intelligent customer support and business workflow automation SaaS

Project goal: Demonstrate a real multi-tenant business platform with secure support workflows, document-grounded AI assistance, human approvals, and auditable background automation. The project must work end to end; no fake AI or fabricated analytics.

Why this is stronger than StockWise AI

- Multi-tenant authorization and organization boundaries, not just single-business CRUD.
- Knowledge-grounded assistance that cites retrieved material and abstains when evidence is insufficient.
- Asynchronous workflows, idempotency, retries, audit trails, and human approval.
- AI quality evaluation, measurable response latency, costs, and failure handling.

Audience & demonstrated business workflow

Small support teams at B2B SaaS firms or online retailers. A customer request becomes a ticket; the platform retrieves authorized knowledge, drafts a response for agent approval, executes an approved action, and records events for reporting.

FIRST RELEASE	ADVANCED RELEASE
Account, workspace, role access; real ticket inbox; lifecycle status; audit events	Tenant-scoped knowledge search; evidence-linked AI drafts; review queue; automation workers; webhook integration

Repository: resolveflow-ai | Primary technologies: React, TypeScript, Express, PostgreSQL, Prisma; Redis + BullMQ and a vector search layer in Phase 2.

ARCHITECTURE

System boundaries and implementation choices

COMPONENT	IMPLEMENTATION / REASON
Frontend	React + Vite + TypeScript; responsive business dashboard, ticket forms, approval interface.
API	Node.js + Express + TypeScript; Zod validation, JWT authentication and role policy.
Primary database	PostgreSQL + Prisma, with workspace ID on tenant-owned records and indexed foreign keys.
AI / retrieval (Phase 2)	Secure ingestion, authorized document chunks, pgvector or supported vector storage, grounded draft answers and source references.
Jobs (Phase 2)	Redis + BullMQ for retries, backoff, idempotency, failure recording, and recoverable processing.
Quality / delivery	API & DB tests, browser E2E tests, CI, structured logs, demo runbook and monitoring.

Trust boundary: every ticket must belong to the current workspace

Browser session → Authenticated user → Workspace membership → Role permission → Tenant-scoped query → PostgreSQL / audit event

Security acceptance criteria

- Users from Workspace A cannot list, search, read, or update Workspace B tickets, documents, events, or AI results.
- VIEWER may read authorized resources but cannot create or modify tickets. Changing roles takes effect on the next protected request.
- Provider keys and database credentials remain server-side. File ingestion has authorization, type and size restrictions.
- Automated external sends require explicit approval until a separately risk-reviewed policy is implemented.

BUILD PLAN

Three development phases

PHASE 1 / FOUNDATION & CONFIGURATION

• Start with .env: copy server and client .env examples; configure PostgreSQL URL, JWT secret, CORS and browser-visible API URL.

• Install dependencies, start local PostgreSQL on port 5433, generate Prisma client, run and commit the initial migration.

• Complete workspace membership and invitations, role authorization, ticket threads, assignment and search/pagination.

• Implement database integration tests for cross-tenant isolation and secure sessions prior to a public release.

• Exit gate: two fictional organizations use the same deployment but cannot access each other's data.

PHASE 2 / AI KNOWLEDGE + WORKFLOW ENGINE

• Ingest approved sample documents, extract/chunk content, create embeddings, and store tenant-scoped searchable passages.

• Return answers as drafts with source citations; refuse unsupported answers; log model, costs and latency.

• Build a human review queue and track edits, approvals, rejections and message delivery outcomes.

• Add Redis/BullMQ jobs and a rules-based automation: priority escalation or owner assignment; include retries and idempotency.

• Exit gate: source-backed draft and approval flow works; job execution is traceable and recoverable.

PHASE 3 / BUGS, UI, DEPLOYMENT & PORTFOLIO

• Fix all functional bugs; improve empty/loading/error states, keyboard support, responsive UI and accessibility.

• Add API integration tests, browser E2E tests, CI, centralized logging, environment validation and release checks.

• Deploy frontend, API, persistent PostgreSQL and background worker with HTTPS, isolated credentials and migrations.

• Seed synthetic demo tickets and documents only; publish video, annotated screenshots, architecture and known limitations.

• Exit gate: independent reviewer can complete the demo without developer assistance.

AI QUALITY

What makes the AI technically defensible

Do not measure success by the number of AI features. Measure whether the system answers supported questions using the correct workspace-owned evidence and appropriately declines unsupported ones.

EVALUATION	WHAT TO TEST
Retrieval hit rate	Does the relevant approved FAQ/document appear in the top results?
Citation support	Can each response claim be checked against an actual retrieved passage?
Abstention	Does the assistant refuse to invent policy when sources are missing or conflicting?
Human acceptance	How often do agents approve a draft without changes? Track rejection causes.
Cost & latency	Tokens, provider spending, request time, failure rate, and retry count per workflow.
Tenant isolation	Do adversarial requests fail to retrieve another organization's knowledge?

Minimum demonstration test set

- 20 straightforward requests answerable from a small, fictional knowledge base.
- 10 ambiguous requests that require follow-up or limited answers.
- 10 unsupported or contradictory requests that should trigger abstention.
- 10 cross-tenant or prompt-injection attempts verifying access boundaries.

Use a frozen test set, write down a rubric, record outcomes and avoid claiming production accuracy from a small synthetic evaluation. Expand coverage before real client use.

Controlled external action example

Customer message → Ticket created → Grounded draft → Agent approval → Job queued → Response delivered → Audit record

GO LIVE

Portfolio assets and project launch criteria

PORTFOLIO ASSET	SPECIFICATION
Live demo	Two fictional demo organizations, predictable accounts and safe sample records.
Screenshots	Workspace dashboard; inbox and ticket detail; AI citations; approval view; workflow run; evaluation metrics.
Walkthrough	90-120 seconds: show real sign-in, two-tenant boundary, live ticket, cited draft, human approval, and automation.
GitHub README	Problem, architecture diagram, stack, setup, real implemented scope, testing steps and limitations.
Engineering proof	Passed CI and E2E runs, test scenarios, roles and data isolation, deployment notes.

Suggested Upwork portfolio title

ResolveFlow AI - Multi-Tenant Customer Support SaaS with RAG & Workflow Automation

What the starter already gives you

- GitHub-ready monorepo and README, .env examples, local PostgreSQL Compose setup.
- Express/TypeScript API, PostgreSQL/Prisma models, account registration, workspace checks and ticket endpoints.
- Responsive React dashboard, ticket forms, list filtering, ticket status actions.
- A small role-policy test suite and a GitHub Actions workflow to expand.

Important distinction: the ZIP is a Phase 1 starter, not a finished production SaaS. Dependencies, database migrations and runtime tests still need to be executed locally. Its first task is .env configuration.

Supporting evidence on market direction

Upwork 2026 research indicates demand for AI integration and workflow automation. Market trends do not guarantee client acquisition.